САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 2: роектирование и технический дизайн API

Выполнил:

Губанов Егор

К3340

Проверил:
Добряков Д. И.

Санкт-Петербург

2026

Задача

1. Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
2. Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
3. Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Необходимо сделать отчёт по шаблону

На защите ожидаю увидеть swagger со спроектированным API и обсудить ваши методы.

Ход работы

1. Описание предметной области

Проектируемый сервис - веб-платформа для аренды недвижимости. Пользователи могут регистрироваться в двух ролях: арендатор (TENANT) или арендодатель (LANDLORD), а также существует роль администратора (ADMIN).

Арендодатель публикует объявления об объектах недвижимости с фотографиями, описанием и условиями аренды. Арендатор находит объекты по фильтрам, просматривает карточку объекта и создаёт заявку на аренду. Стороны ведут переписку по объекту. Сделка проходит статусы: PENDING → ACTIVE → COMPLETED.

API проектируется на основе ER-модели (ДЗ1), включающей сущности: User, PropertyType, Property, Photo, Deal, Message.

2. Функциональные требования

Требование	**Описание**
Регистрация	Создание учётной записи: имя, фамилия, email, пароль, роль (TENANT / LANDLORD / ADMIN).
Вход	Пользователь передаёт email и пароль, получает Bearer-токен и профиль без пароля.
Личный кабинет	Как арендатор: список сделок, где пользователь - арендатор. Как арендодатель: мои объекты и сделки по ним (отдельные методы).
Поиск недвижимости	Каталог опубликованных объектов с фильтрами: тип, город, мин./макс. цена. Постраничный вывод (page, page_size).

Страница объекта	Карточка объекта: описание, условия аренды, цена, адрес, владелец (кратко), галерея фото.
История	Сводный метод: сообщения пользователя и его сделки одним JSON для экрана истории.
CRUD объявлений и фото	Владелец может создавать, редактировать и удалять объявления. Добавление и удаление фото по URL.
Сделки	Арендатор создаёт заявку на аренду. Владелец объекта меняет статус (PENDING → ACTIVE → COMPLETED).
Сообщения	Переписка по объекту между арендатором и арендодателем. Отметка прочтения.

Табл. 1.

3. Нефункциональные требования

Требование	**Описание**
Безопасность	Пароль не возвращается в ответах. В продакшене хранится хэшем. Доступ к закрытым методам по заголовку Authorization: Bearer <token>. Транспорт - HTTPS.
Единый формат ошибок	Тело любой ошибки содержит числовой code (HTTP-код) и строку message.
Пагинация	Все списковые методы поддерживают параметры page и page_size. Ответ содержит total.
Документирование	Спецификация OpenAPI 3.0. Просмотр через локальный Swagger UI после сборки.
Формат данных	JSON, кодировка UTF-8.

Табл. 2.

4. Выбор формата API: REST

Для реализации выбран REST API (ресурсы и HTTP-методы).

Критерий	**REST API (выбрано)**	**BFF**
Клиентов	Один тип (веб)	Несколько разных
Структура ресурсов	Совпадает с таблицами БД	Агрегация из нескольких сервисов
Тестирование	Postman / Swagger	Сложнее
Сложность	Меньше	Дополнительный слой

Табл. 3.

Вывод: в данном проекте один бэкенд и один тип клиента (веб-приложение); сущности API соответствуют таблицам БД, что упрощает реализацию в Express + ORM и тестирование через Postman и Swagger. BFF целесообразен при нескольких сильно различающихся фронтах и агрегации из нескольких сервисов - в данном случае это избыточно.

Префикс всех путей: /api/v1.

5. Описание эндпоинтов API

Полный список эндпоинтов с указанием метода, пути, назначения и необходимости авторизации:

Метод	Путь	Назначение	Авторизация
POST	`/api/v1/auth/register`	Регистрация нового пользователя	нет
POST	`/api/v1/auth/login`	Вход, получение токена	нет
GET	`/api/v1/me/`	Данные текущего пользователя	Bearer
GET	`/api/v1/me/renting`	Сделки, где я - арендатор	Bearer
GET	`/api/v1/me/owning`	Мои объекты и активные сделки по ним	Bearer
GET	`/api/v1/me/owning/deals`	Все сделки по моим объектам	Bearer
GET	`/api/v1/property-types/`	Справочник типов недвижимости	нет
GET	`/api/v1/properties/`	Поиск и список (фильтры в query)	нет
GET	`/api/v1/properties/{id}`	Карточка объекта	нет
POST	`/api/v1/properties/`	Создать объявление	Bearer (владелец)
PUT	`/api/v1/properties/{id}`	Обновить объявление	Bearer (владелец)
DELETE	`/api/v1/properties/{id}`	Удалить объявление	Bearer (владелец)
POST	`/api/v1/properties/{id}/photos`	Добавить фото по URL	Bearer (владелец)
DELETE	`/api/v1/photos/{id}`	Удалить фото	Bearer (владелец)
POST	`/api/v1/deals/`	Создать заявку на аренду	Bearer (арендатор)
GET	`/api/v1/deals/{id}`	Получить сделку	Bearer (участник)
PATCH	`/api/v1/deals/{id}`	Сменить статус сделки	Bearer (владелец объекта)
GET	`/api/v1/messages/`	Список сообщений пользователя	Bearer
POST	`/api/v1/messages/`	Отправить сообщение	Bearer
PATCH	`/api/v1/messages/{id}/read`	Пометить сообщение прочитанным	Bearer

GET	/api/v1/history/	Сообщения и сделки для экрана истории	Bearer

Табл. 4.

Спецификация OpenAPI генерируется из TypeSpec-исходников командой npm run build и сохраняется в файл homeworks/hw2/tsp-output/schema/openapi.yaml.

6. Форматы тел запросов и ответов

6.1. Регистрация

Запрос: POST /api/v1/auth/register

Тело запроса (JSON):

```
{
  "email": "ivan@mail.ru",
  "password": "secret123",
  "first_name": "Иван",
  "last_name": "Иванов",
  "role": "TENANT"
}
```

Ответ 200 - схема AuthOk:

```
{
  "access_token": "<JWT>",
  "token_type": "Bearer",
  "user": {
    "id": "uuid",
    "role": "TENANT",
    "first_name": "Иван",
    "last_name": "Иванов",
    "email": "ivan@mail.ru",
    "is_verified": false,
    "created_at": "2026-03-29T12:00:00Z",
    "updated_at": "2026-03-29T12:00:00Z"
  }
}
```

Поле password в ответе отсутствует.

6.2. Поиск объектов недвижимости

Запрос: GET /api/v1/properties/?city=Санкт-Петербург&min_price=10000&max_price=50000&page=1&page_size=20

Ответ 200 - схема PropertyListResponse:

```
{
  "items": [
    {
      "id": "uuid",
      "title": "Квартира в центре",
      "city": "Санкт-Петербург",
      "price": 35000,
      "type": { "id": "uuid", "title": "Квартира" }
    }
  ],
  "total": 42
```

```
}
```

6.3. Создание сделки

Запрос: POST /api/v1/deals/

Тело запроса (JSON):

```
{
  "property_id": "uuid",
  "start_date": "2026-04-01T00:00:00Z",
  "end_date": "2026-04-30T00:00:00Z"
}
```

Ответ 200 - схема Deal с полями id, property_id, tenant_id, status (PENDING), start_date, end_date, total_price, created_at, updated_at.

7. Система ошибок API

Все ошибки возвращаются в едином формате:

```
{
  "code": 404,
  "message": "Объект недвижимости не найден"
}
```

Таблица возможных ошибок:

HTTP-код	Тип (OpenAPI)	Когда возникает
401	`UnauthorizedError`	Нет токена, неверный логин или пароль
403	`ForbiddenError`	Недостаточно прав: не та роль или не владелец ресурса
404	`NotFoundError`	Объект, сделка, сообщение или фото не найдены
409	`ConflictError`	Конфликт: email уже занят, недопустимый переход статуса
422	`ValidationError`	Невалидное тело запроса или параметры
500	`ServerError`	Внутренняя ошибка сервера

Табл. 5.

В спецификации OpenAPI определены отдельные схемы ошибок: UnauthorizedError, ForbiddenError, NotFoundError, ConflictError, ValidationError, ServerError - все наследуют общую форму с полями code и message. Описания ошибок в поле message представлены на русском языке.

8. Средства реализации

Спецификация API разработана с использованием следующих инструментов:

- TypeSpec - язык описания API-контракта от Microsoft. Позволяет определять модели, интерфейсы и операции с последующей компиляцией в OpenAPI-схему. Исходники расположены в homeworks/hw2/: main.tsp, models (rental.tsp, error.tsp, auth_scheme.tsp), interfaces (auth.tsp, me.tsp, property_types.tsp, properties.tsp, photos.tsp, deals.tsp, messages.tsp, history.tsp).

- OpenAPI 3.0 - стандарт описания REST API. Схема генерируется командой npm run build (tsp compile) и сохраняется в homeworks/hw2/tsp-output/schema/openapi.yaml.
- Swagger UI - веб-интерфейс для просмотра и интерактивного тестирования OpenAPI-спецификации. Поднимается локально через Express + swagger-ui-express.
- Node.js + Express - используются только для сервинга Swagger UI. Реализация бэкенда планируется в следующей лабораторной работе.

9. Порядок запуска просмотра документации

Для просмотра Swagger UI необходимо перейти в каталог homeworks/hw2 и выполнить следующие команды:

```
npm install
npm run build
npm start
```

После запуска документация доступна в браузере по адресу: http://localhost:8000/docs

Вывод: в ходе выполнения домашней работы были сформулированы функциональные и нефункциональные требования к сервису аренды недвижимости, обоснован выбор REST API, спроектированы 21 эндпоинт в формате OpenAPI 3.0. Спецификация охватывает все ключевые сценарии: аутентификацию, управление объявлениями и фотографиями, создание и ведение сделок, переписку и просмотр истории. Реализована единая система ошибок с кодами 401, 403, 404, 409, 422, 500. Схема генерируется из TypeSpec-исходников и доступна для просмотра через Swagger UI локально. Реализация бэкенда (Express, PostgreSQL, JWT-аутентификация) планируется в следующей лабораторной работе.

Список использованных инструментов

1. TypeSpec (microsoft/typespec) - язык проектирования API-контрактов.
2. OpenAPI 3.0 - стандарт описания REST API.
3. Swagger UI - инструмент для визуализации и тестирования OpenAPI-спецификации.
4. Node.js + Express + swagger-ui-express - локальный сервер для хостинга документации.